

졸업 프로젝트 7월 2차 격주보고서		
SNN을 활용한 고효율 추천 시스템 개발 및 최적화 연구		
2020058259 이승현 2020061676 이강혁		지도교수 김상욱 교수님
지난 2주간 진행사항	1. NCF 모델 구현 - 코드 간결화 및 최적화를 통해 HDCG@K 0.75까지 목표, 여러 평가지표 구현 및 작성 - 그 후 dataset을 implicit data로 변경 2. SNN 활용한 추천시스템 구현에 대한 내용을 정리 후, 시뮬레이터를 제공하고 그 시뮬레이터를 neuromorphic chip을 사용하여 대신 실행해주는 사이트에 제출 3. 논문 찾아보며 SNN의 구조 공부 - 활성화 함수, loss, 데이터 인코딩 등 세부적인 부분 자세히 전력 비교 적용 방법 공부	△ △ ○
진행 내용	SpikingJelly를 통한 SNN 구조의 전력 효율 분석과 실험 설계 1) 소개 SpikingJelly는 PyTorch 기반으로 딥러닝 모델과 유사한 학습 및 실험 환경을 제공하며, surrogate gradient 기반의 SNN 학습이 가능하고, spike count 및 뉴런 발화 패턴을 정량적으로 추출할 수 있어 이론적 전력 분석에도 적합합니다. 또한 다양한 뉴런 모델(IF, LIF 등)과 시간 축 기반의 시뮬레이션 기능을 지원하여, 추천 시스템 구조 내에서 시간-이벤트 기반 계산의 효과를 실험적으로 검증할 수 있습니다.	

2) 뉴로모픽 특성을 반영한 시뮬레이터

SpikingJelly는 단순 PyTorch wrapper가 아닌 시간 기반 누적, spike event 처리, surrogate 학습 등 SNN 고유 특성을 내장한 시뮬레이터입니다.

Jelly에서 측정 가능한 spike count, 뉴런 발화율, timestep dynamics는 실제 뉴로모픽 칩에서도 성능에 영향을 주는 핵심 지표입니다. 따라서 Jelly 기반 전력 분석은 단순한 이론적 가정이 아니라, 하드웨어 이식 가능성을 반영한 정량적 평가 방식이 됩니다.

3) 현실적인 제약과 실험 일관성의 필요성

뉴로모픽 칩은 연구자들에게 범용적으로 열려 있지 않으며, 사용을 위한 접근, 환경 설정, 커스텀화에 큰 제약이 따르게 됩니다. 또한 ANN-to-SNN 변환을 전제로 실험할 경우 구조의 설계 자유도가 제한되며, 구조적 아이디어 자체를 평가하기 어렵습니다.

반면 Jelly는 설계, 학습, 평가, 에너지 분석까지 일관된 실험 환경을 제공하고 결과 비교의 신뢰성을 높입니다. 따라서 Jelly 기반 분석은 왜곡 없이 구조 자체의 효율성과 작동 메커니즘을 정밀하게 분석할 수 있게 됩니다.

논문 정리

1. ANN-to-SNN 방식 논문

논문 소개:

Kumar, Raghavendra & Perera, Darshika. (2023). Neuromorphic Sentiment Analysis Using Spiking Neural Networks. Sensors. 23. 10.3390/s23187701.

1) 소개

대부분의 연구는 ANN 또는 SNN 단일 구조에 집중하거나, 변환 방식을 소개하더라도 실질적인 뉴로모픽 하드웨어 상의 적용 사례는 드뭅니다. 반면 본 논문은 기존 ANN의 높은 정확도를 유지하면서, SNN의 에너지 효율성을 함께 달성하는 감성 분석 모델을 제안합니다. 모델은 학습단계에서는 ANN을 활용하고, 추론 단계에서 SNN을 사용함으로써 안정적인 학습 성능과 실시간 에너지 효율성을 확보합니다. 학습된 ANN을 변환하여 실제 SpiNNaker 하드웨어에 배포하고, 감성 분석 작업을 실시간으로 수행함으로써 모델의 실행 가능성과 효율성을 입증하였습니다.

2) ANN-SNN 변환 방법

ANN에서 SNN으로의 변환은 Rueckauer와 Liu(2021)가 제안한 방법을 기반으로 합니다. 이 방식은 ANN에서 사용된 ReLU 뉴런을 IF 뉴런으로 치환하면서 학습된 가중치는 그대로 유지하는 방식입니다. 시뮬레이션 과정에서 SNN의 뉴런은 일정한 시간 단위를 기준으로 스파이크를 발화하며, 그 평균 발화율이 ANN에서의 뉴런 활성화 값에 근사하도록 설계합니다.

3) SpiNNaker-하드웨어 플랫폼

SpiNNaker는 저전력, 고효율 프로세서들이 네트워크로 연결된 대규모 병렬 시스템입니다. 네트워크의 각 노드는 소수의 뉴런과 시냅스를 실시간으로 시뮬레이션 또는 실행하는 역할을 하며, 서로 통신하며 계산 부하를 분산합니다.

해당 논문에서는 PyNN 인터페이스를 활용하여 SpiNNaker 아키텍처에서 SNN을 실행합니다.

4) 구현 구조

1. Keras 기반의 ANN 모델 생성

- 입력 정보를 추출해 기본 구조를 생성합니다.

2. ANN → SNN 변환

- ANN의 가중치를 그대로 사용하고, 아날로그 뉴런을 IF 스파이킹 뉴런으로 교체합니다.

3. SNN을 시뮬레이터/하드웨어에 내보내기

- 변환된 SNN은 시뮬레이터(INI 또는 SpiNNaker) 또는 실제 하드웨어에서 실행됩니다.

4. 최종 평가 및 비교

- SNN 모델을 테스트하고, 정확도와 에너지 소비를 기존 ANN 모델과 비교합니다.

이때, 뉴런 모델은 LIF 같은 비선형 신경모델을 사용하며, 시간에 따라 신호를 전달하는 방식입니다.

5) 결론

이 모델은 기존 ANN에 비해 상당한 에너지 절감 효과를 보여주었습니다. 결과적으로 ANN의 학습 능력과 SNN의 에너지 효율성을 모두 결합한 성공적인 사례로 평가됩니다.

2. SNN학습 성능/전력 비교 논문

논문 소개:

Man, Yao & Zhao, Guangshe & Zhang, Hengyu & Yifan, Hu & Tian, Yonghong & Xu, Bo & Li, Guoqi. (2023). Attention Spiking Neural Networks. IEEE Transactions on Pattern Analysis and Machine Intelligence. PP. 1-18. 10.1109/TPAMI.2023.3241201.

1) 소개

SNN은 생물학적 뇌를 모방하여 이벤트 기반 처리와 에너지 효율에 뛰어난 특성을 가집니다. 하지만 전통적인 ANN에 비해 정확도 면에서 열세였으며, 특히 대규모 데이터셋에서의 표현력 부족이 문제입니다. ANN-to-SNN 변환 방식은 정확도를 유지할 수 있지만, 수천 개의 timestep이 필요해 latency와 에너지 소모가 커지는 한계가 있습니다. 이를 해결하기 위해 이 논문은 attention 메커니즘을 SNN에 적용하여, 적은 timestep으로도 고정확도 및 고효율을 동시에 달성하는 방법을 제안합니다.

2) 제안 방법: Multi-dimensional Attention SNN (MA-SNN)

본 논문은 Temporal, Channel, Spatial Attention을 동시에 적용하는 MA (Multi-dimensional Attention) 모듈을 제안합니다. 이 attention은 각 뉴런의 membrane potential에 직접 작용하여, 중요한 정보는 강조하고 불필요한 background spike는 억제합니다.

이런 방식은 spike 발생 확률을 효과적으로 조절하며, network의 전체 spike activity를 감소시켜 energy efficiency를 크게 향상시킵니다. 또한 MA 모듈은 기존 SNN 구조 위에 간단히 추가할 수 있는 plug-and-play 형태로 설계되어, 구조 변경 없이 성능 향상을 가능하게 도와줍니다.

3) 학습 및 구현 방법

모든 실험은 PyTorch 기반으로 GPU에서 수행되며, SNN은 surrogate gradient를 활용한 직접 학습 방식을 사용했습니다.

뉴런 모델은 LIF를 사용하며, spike는 surrogate 함수로 근사해 gradient를 전달합니다. 입력은 정적 이미지일 경우 timestep 수만큼 반복 복제하고, 이벤트 기반 데이터는 frame-based 방식으로 전처리됩니다. 마지막으로 정확도는 마지막 timestep의 membrane potential이나 누적 spike 수를 softmax를 통해 분류하여 계산합니다.

4) 성능 및 비교 실험

ImageNet-1K, DVS128 Gesture/Gait 등의 벤치마크에서 실험이 진행되었습니다.

특히 ResNet-104 기반 SNN에 MA 모듈을 삽입한 결과, timestep을 단 1~4로 유지하면서도 ANN 수준 또는 이상의 정확도를 달성하였습니다.

MA-SNN은 ResNet-104 구조에서 timestep = 1로 75.92%, timestep = 4로 77.08% 정확도를 기록하며, 기존 ANN(76.87%)과 ANN-to-SNN 방식(VGG-16 @ T=2048, 75.32%)을 초과하는 성능을 보였습니다.

5) 저전력 효율 측정 방식

논문에서의 저전력은 실제 전력 측정이 아닌, 연산량 기반의 이론적 추정을 통해 계산됩니다. 각 층의 MAC 및 AC 연산량에 대해 단위 에너지 값을 곱해 총 전력을 계산하며, 이를 통해 baseline 대비 에너지 효율 비율인 rEE(Relative Energy Efficiency)를 산출합니다.

따라서 전력 효율 결과는 추론 단계 기준이며, 학습 시 에너지는 고려되지 않습니다.

6) 결론 및 의의

이 논문은 기존의 ANN-to-SNN 변환 방식이나 복잡한 구조 없이, 단순한 attention 메커니즘을 도입함으로써 SNN의 정확도, 에너지 효율, latency 문제를 동시에 해결할 수 있음을 보였습니다.

특히 attention은 spike를 낭비하지 않도록 하여 적은 spike로도 많은 정보를 전달하게 만들며, 이는 Timestep을 늘리는 대신 스파이크의 질을 높이는 방식으로 구조적, 계산적 비용 없이 SNN을 강화하는 매우 실용적인 전략입니다.

SNN의 구조

1) Surrogate 함수

- 실수 데이터를 이산 스파이크로 변환합니다. 이때 스파이크 함수가 미분 불가능하므로, 이계도 수준으로 sigmoid gradient를 근사하는 "Surrogate" 기법을 사용합니다.
- def forward
 - input tensor의 각 원소가 0보다 크면 1, 아니면 0으로 출력→ 이산 스파이크
 - 이산 스파이크 값을 class 객체에 저장
- def backward
 - 저장한 이산 스파이크 값을 로드해서 $\beta \cdot \text{sigmoid}(\beta x) \cdot (1 - \text{sigmoid}(\beta x))$ 형태의 근사미분값을 계산

2) LIFLayer 클래스

- Leaky Integrate-and-Fire: 실제 신경의 동작 방식을 회로이론으로 모방하여 계산합니다. 이전 뉴런들의 spike를 통합하였을 때의 전위와 시간의 흐름에 따른 전위의 leaky를 고려하여 특정 임계치를 넘는지 확인하여 현재 뉴런의 spike를 결정합니다.
- parameter
 1. tau: 막전위 누설 시상수 (leaky)
 2. threshold: 임계치
 3. mem: 각 뉴런의 막전위
- def forward
 - 학습 단계마다 각 뉴런의 전위를 mem 변수에 저장
 - 입력 전류 $I = f_c(x)$ 에 대해 오일러 적분하여 v (전위차) 계산:
$$dv = \frac{I-v}{\tau}, \quad v \leftarrow v + dv$$
 - $v > \text{threshold}$ 인 위치에 스파이크 발생 (Surrogate 함수를 통해 1로 변환)
 - 발화한 뉴런의 v 를 0으로 초기화 (mem 변수 초기화)

3) SNN 모델

- Data Embedding → LIF Hidden Layer → LIF Output Layer → prediction (timesteps에 대한 spikes 비율)
- parameter
 - timesteps: 매 샘플에 대해 SNN 학습이 동작할 횟수
- def forward
 - timesteps 동안(timesteps만큼 loop) LIF Hidden Layer → LIF Output Layer를 순회하며 학습 진행
 - timestep마다 발생하는 spike 수를 누적하여 저장
 - 누적 spike 수 /timesteps를 결과로 리턴

4) Training & Evaluation

- Loss function: BCEWithLogitsLoss 사용하여 확률값으로 변환
- Optimizer: 일반적인 ANN과 동일하게 Adam 사용

전력 비교

에너지 효율성 비교

- 연산량: SNN-누적 spike 수 /timesteps
- 메모리 접근 횟수: SNN-각 뉴런의 막전위 저장 횟수 (timesteps에 비례)
- 메모리 재사용 정도: ANN의 메모리 재사용성

참고한 논문에서는 memory access를 고려한 전력 비교 방식을 제안하고 있습니다. 이는 SNN과 ANN의 비교에서 동일한 이벤트 기반 아키텍처를 가정하고, 하드웨어가 아닌 디지털 환경에서의 비교를 전제로 합니다.

SNN의 한 layer에서 총 에너지(결과값)은 layer에 존재하는 시냅스에 input으로 들어온 모든 spike의 개수와 뉴런에 input으로 들어온 spike를 timestep에 따라 가중하여 계산한 전위 값의 합입니다. 즉, 위 SNN 모델의 결과값, 누적 spike 수 /timesteps가 됩니다.

Timestep마다 위 LIFLayer 클래스의 mem 변수의 spike, 상태 변수 등의 값이 업데이트 되며 메모리 접근이 일어납니다. 일반적으로 시냅스 연산 에너지가 timestep 업데이트 에너지에 비해 크기 때문에 각 timestep에서의 업데이트 에너지는 시냅스 연산 에너지에 비해 무시할 수 있지만, timesteps의 크기가 큰 경우에는 이를 고려하여야 합니다. 일반적으로 (timesteps= 10)에서 뉴런 업데이트와 관련된 오버헤드를 무시할 수 있고, (timesteps = 500)에서는 중요한 고려 사항이 됩니다.

	Timesteps가 충분히 작은 경우, 각 timestep에서 업데이트되는 SNN 모델의 상대적인 에너지 효율은 뉴런 개수에 대한 시냅스 개수의 비율(connection / node), spike sparsity에 따라 달라집니다. 이 비율이 0.5를 넘으면 SNN은 ANN보다 전력 효율이 떨어집니다. 그러나 비율이 감소함에 따라 SNN의 에너지 효율성은 빠르게 증가합니다. 이를 통해 이론적으로 spike sparsity를 구하여 SNN의 상대적인 전력 효율을 비교할 수 있음을 알 수 있었습니다. 그러나 비율이 감소함에 따라 SNN의 에너지 효율성은 빠르게 증가합니다. 이를 통해 이론적으로 spike sparsity를 구하여 SNN의 상대적인 전력 효율을 비교할 수 있음을 알 수 있었습니다.
진행 중 이슈	SNN 프레임워크 선정에 대한 고민
다음 2주간 계획	1. NCF 모델 구조 개선 및 최적화 2. SNN 기본모델 구현 3. alibaba 데이터 전처리